Creating A Database for Exoplanet Analysis

Group 6: Zoë DeFord, Lennart Kindt, Dawud Trebo /
INF21306 / April 2026

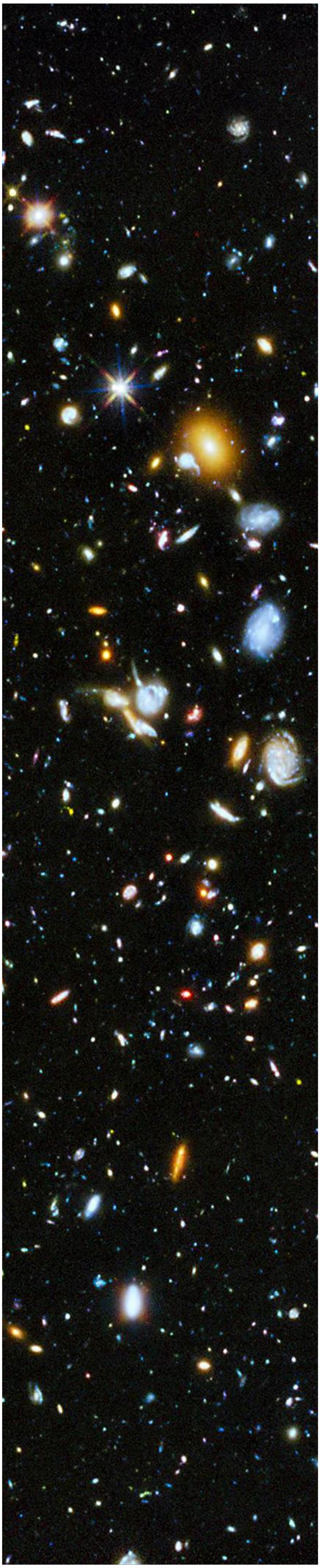
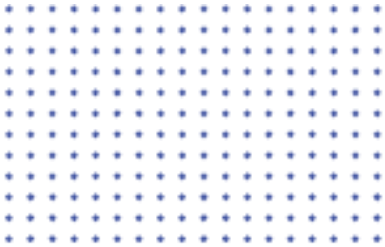


TABLE OF CONTENTS

1. INTRODUCTION



Since 1992, the discovery of exoplanets has skyrocketed. Often, the data for these exoplanets and their solar systems are gathered by a variety of institutions by a wide range of methods. The data is then compiled for NASA but is unorganized and hard to read. NASA provides this large, comprehensive database in their exo-planet archive which is useful for data gathering but not succinct enough for thorough analysis of the data. A well-functioning, readable database is necessary for proper analysis of the exoplanets to draw meaningful scientific conclusions from decades of accumulated data.

Background

The information provided on exoplanets is quite vast. Research on exoplanets involves analyzing multiple factors such as planetary radius, surface temperature, orbital characteristics, and properties of the host star. This information is conveyed in a wide variety of ways. The goal of this database is to sift through the meaningful data that can provide insight on which planets have these characteristics logged and provide a comprehensive overview that can be easily read and interpreted. The interpretation will include examining attributes that can provide insight into habitability of these exoplanets.

Aim

Topics we want to research:

Discovery

- When did we discover the most planets?
- What was the most common discovery method and is there a relationship between method and year?
- Which facilities discovered the most planets?

Planetary Attributes

- Can we rank the planets by similarity to earth?
- Which planets are the hottest and does this correlate with how far it is from the host star?
- Which planets could potentially sustain life based on their attributes?

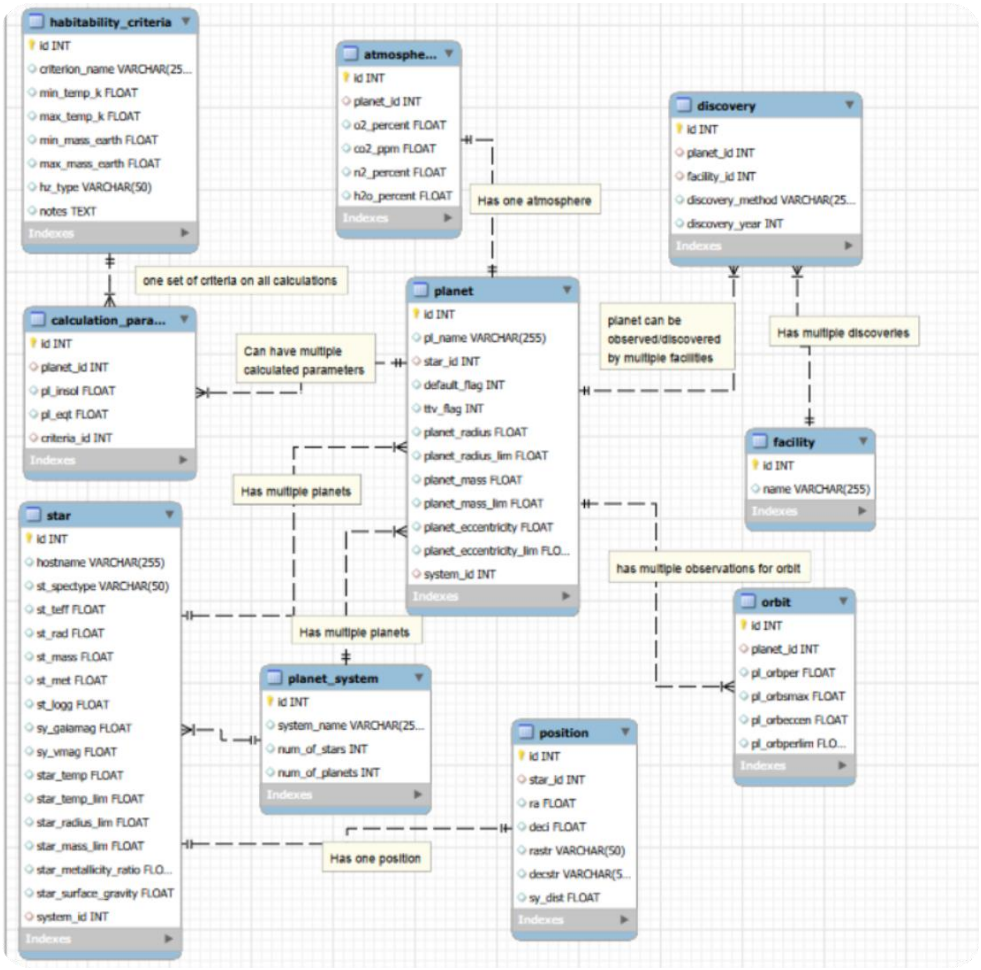
Habitability using Advanced Queries in Python

- How can python be used to create a visualization of the planets that are most likely to be habitable?

Data

NASA Exoplanet Archive from NASA Exoplanet Science Institute (NESI):
Https://exoplanetarchive.ipac.caltech.edu/cgi-bin/TblView/nph-tblView?app=ExoTbls&config=PS
A small subset of this data was gathered for efficacy. They were chosen at random by name.

2. ER DIAGRAM



Entities of DB

This database was designed following normalization principles to reduce redundancy and ensure data consistency. Each table represents a distinct real-world entity (such as planets, stars, and discovery events), while relationships between tables are implemented using foreign keys. One-to-many relationships are used where a single entity can have multiple related records (one star can have multiple planets), and one-to-one relationships are used for attributes that belong uniquely to a single entity (a star's position).

1 Entity *star*

The *star* table is a central entity for the ER diagram. It contains all the host stars that the exoplanets orbit around. Each star can have multiple planets, and the stars have several characteristics such as temperature, magnitude, radius, mass, metallicity, surface gravity, and spectral type. Each star can have multiple planets orbiting it, but each planet belongs to exactly one star. This relationship is implemented using the foreign key planet.star_id, which references star.id. This ensures that every planet is always linked to a valid star.

2 Entity *planet*

The *planet* contains all the information on the exoplanets in the database and a core component of our data and analysis. Each planet has an ID, a name, the discovery information, and the physical properties such as mass, radius, eccentricity, etc. Each planet has other properties that are expressed in other tables, such as atmosphere, orbit, and discovery-facility.

3 Entity *discovery*

The discovery of a planet is modelled as a separate entity to avoid redundancy and allow multiple observations. The discovery table connects planets and facilities and stores additional information, such as discovery method and year. This creates a many-to-many relationship between planets and facilities, where one planet can be observed by multiple facilities and one facility can discover multiple planets.

4 Entity facility

As previously explained, the facility table is connected to the planet table via the discovery table. It only stores the facility name. A facility (such as a telescope or observatory) can be involved in multiple discovery events. Each discovery record references one facility through discovery.facility_id, creating a one-to-many relationship.

5 Entity *orbit*

A planet can have multiple orbit records because orbital parameters may be measured multiple times or updated over time. The orbit table stores these measurements, and each record references a planet via orbit.planet_id. This allows the database to preserve historical and observational variations.

6 Entity *star_position*

Each star has a single spatial position defined by its right ascension and declination. This is modeled as a one-to-one relationship, where the `position` table contains exactly one record per star, linked via `position.star_id`.

7 Entity *atmosphere*

Each planet can have one associated atmospheric composition record, stored in the `atmosphere` table. This includes gas percentages such as oxygen, carbon dioxide, and nitrogen. The relationship is one-to-one and implemented using `atmosphere.planet_id`.

8 Entity *planet_system*

A planetary system can contain multiple planets, while each planet belongs to one system. The table `planet_system` stores system-level properties such as the number of stars and planets. The relationship is implemented through `planet.system_id`, which references `planet_system.id`.

9 Entity *habitability_criteria*

Habitability criteria define thresholds such as temperature and mass limits. These criteria are stored in a static table and applied to many planets. The relationship is implemented through `calculation_params.criteria_id`, linking each calculation to a specific set of criteria (for example: conservative or optimistic).

10 calculation parameters

A planet can have multiple calculated parameters depending on different habitability models. The `calculation_params` table stores values such as insolation flux and equilibrium temperature. Each record references a planet via `planet_id`, allowing multiple calculations per planet.

3. SQL QUERIES

1. List planets ordered by temperature (hottest first) & group by distance from star

The aim of this query was to find planets with the highest temperature and see if there is a correlation with distance to the host star? JOIN is used to join the two tables planet and orbit, so that the relevant information is in one table. Then ORDER BY is used to order the data for easier reading. This information can later be used to examining attributes that can provide insight into habitability of the exoplanets.

```
-- 1. Planets ordered by distance from star
 SELECT planet.pl_name, orbit.pl_orbsmax FROM planet JOIN orbit ON plane
t.id = orbit.planet_id ORDER BY orbit.pl_orbsmax ASC;
```

2. Number of planets per discovery method & the most common year of discovery (is there relationship between method and year?)

This query looks at the amount of planets found per discovery method, and at the relationship between discovery method and time (in years). First COUNT(*) is used for counting planets , these will be then saved AS num_planets. Then GROUP BY is used for grouping by discoverymethod. This information can give insight into which methods work best,

```
- 2. Planets per discovery method & year
SELECT discoverymethod, disc_year, COUNT(*) AS num_planets FROM plan
et GROUP BY discoverymethod, disc_year ORDER BY num_planets DESC;
```

3. Number of planets discovered per year

The next query uses the commands COUNT(*), GROUP BY and ORDER BY to make the data from planet more comprehensive. Now in one glance the discovery year and the amount discovered can be seen, it becomes easily clear which year was most fruitful in terms of exoplanet discovery.

```
- -3. Planets per discovery method and most common
SELECT disc_year, COUNT(*) as amount_y
FROM planet
group by disc_year
ORDER BY amount_y DESC
```

4. Rank planets by similarity to Earth (ORDER BY)

The aim of this query was to rank planets by their similarity to Earth. To find the most suitable candidates and also rank them. JOIN was used to join star ON planet.star WHERE star.st_teff was BETWEEN 4000 AND 7000 AND orbit.pl_orbsmax was BETWEEN 0.8 AND 1.5. These specific values were

star.st_teff is the temperature of the star and orbit.pl_orbsmax was the longest radius of an elliptic orbit of an exoplanet help in finding potential habitable planets (zones).

```
-- 4. Rank planets by similarity to Earth (using orbit distance as proxy for habitable range)
SELECT planet.pl_name, orbit.pl_orbsmax, orbit.pl_orbeccen FROM planet JOIN orbit ON planet.id = orbit.pla
net_id JOIN star ON planet.star_id = star.id WHERE star.st_teff BETWEEN 4000 AND 7000 AND orbit.pl_orbsm
ax BETWEEN 0.8 AND 1.5 ORDER BY orbit.pl_orbsmax ASC;
```

5. Create system view

This query creates a view of the star systems. A view gives an overview of relevant information, in this case we SELECT all data of planets and stars and JOIN them based on their id. What results in an easy to read view of all star systems which can be used to see in one glance which planet belongs to which star.

```
--5. Create a view of the planet/star system
CREATE VIEW StarSystemView AS
SELECT
   s.star_id,
   s.star_name,
   p.planet_id,
   p.planet_name,
   pp.radius,
   pp.temperature
FROM Star s  JOIN Planet p ON s.star_id = p.star_id  JOIN PlanetPhysical pp
ON p.planet_id = pp.planet_id;
```

6. Facilities that discovered the most planet

The following query creates an table where facilities are ordered by most planets found. Planets are COUNT(*) AS num_planets then tables are joined and ordered by highest (DESC). This table can be used to see highest ranking facility's in terms of exoplanets found. ESA can use this list to contact the facility's and ask information about the strategies/methods used to find so many planets.

```
-- 6. Facilities that discovered the most planets
SELECT facility.name, COUNT(*) AS num_planets FROM planet JOIN facility ON planet.facility_id = facility.id
GROUP BY facility.name ORDER BY num_planets DESC;
```

7. Planets in habitable zone (where liquid water is possible)

The last query will give an list of all planets that are in the habitable zone (where liquid water is possible). This happens by selecting the relevant information joining based on the id's WHERE star temperature is BETWEEN 4000 AND 7000 AND longest radius of an elliptic orbit is BETWEEN 0.5 AND 2.0. Liquid water presence on a planet is an important indication for livability, because for biological processes liquid water is required, at least for humans. So this table will give a list of likely candidate planets where habitability is possible.

```
-- 7. Planets in habitable zone (liquid water possible)
```

SELECT planet.pl_name, orbit.pl_orbsmax, star.st_teff FROM planet JOIN orbit ON planet.id = orbit.plane
t_id JOIN star ON planet.star_id = star.id WHERE star.st_teff BETWEEN 4000 AND 7000 AND orbit.pl_orbs
max BETWEEN 0.5 AND 2.0;

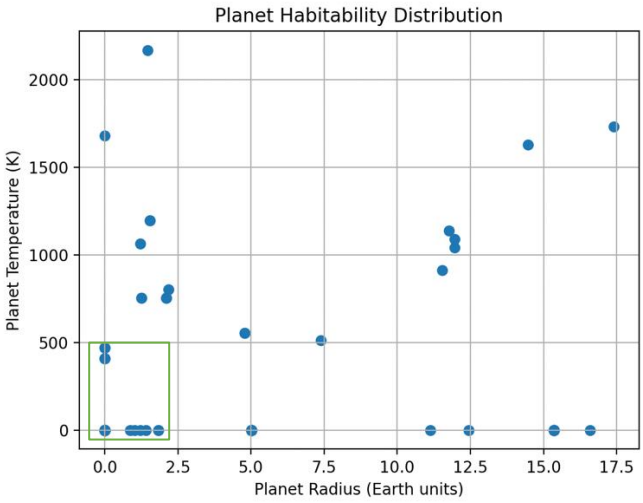
4. ADVANCED FUNCTIONS

To advance the functions of the SQL database beyond the standard queries, a python analysis was developed. This tool connects directly to the MySQL database through PyCharm and computes the total habitability for each planet using key planetary and stellar attributes. This helps answer our research question in a more comprehensive way.

The habitability score is based on the similarity of a planet to Earth, using:

- Planet temperature (`pl_eqt`)
- Planet radius (`pl_rade`)
- Host star temperature (`st_teff`)

The results were visualized using a scatter plot, allowing for identification of planets that fall within habitable ranges. This approach enables more advanced analysis than SQL alone, as it combines computation, ranking, and visualization in a single workflow.




```python
import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt

# connect to your database
conn = mysql.connector.connect(
    host="data-management-server-1.cbxlqxkadwhn.us-east-1.rds.amazonaws.com",
    user="ZoeDeford",
    passwd="a5f05195058ed46c2a13a9beceddd091233cf171",
    db="group06db",
    port=3306
)

# query
query = """
SELECT
    p.pl_name,
    s.hostname,
    cp.pl_eqt,
    p.planet_radius,
    s.star_temp
FROM planet p
JOIN star s ON p.star_id = s.id
JOIN calculation_params cp ON p.id = cp.planet_id
WHERE cp.pl_eqt IS NOT NULL
AND p.planet_radius IS NOT NULL
AND s.star_temp IS NOT NULL;
"""

# load data
df = pd.read_sql(query, conn)

# close connection (important)
conn.close()

# habitability score
df["score"] = (
    abs(df["pl_eqt"] - 288) * 0.5 +
    abs(df["planet_radius"] - 1) * 0.3 +
    abs(df["star_temp"] - 5778) * 0.2
)

# top 20 planets
df_sorted = df.sort_values("score").head(20)

print("\nTop 20 Most Habitable Planets:\n")
print(df_sorted[["pl_name", "hostname", "score"]])

# plot
plt.figure()
plt.scatter(df["planet_radius"], df["pl_eqt"])
plt.xlabel("Planet Radius (Earth units)")
plt.ylabel("Planet Temperature (K)")
plt.title("Planet Habitability Distribution")
plt.grid(True)
plt.show()
```